Effect Systems & Interpreters

nterpreters · Read file → call service → write result; prod vs test interp · Build with

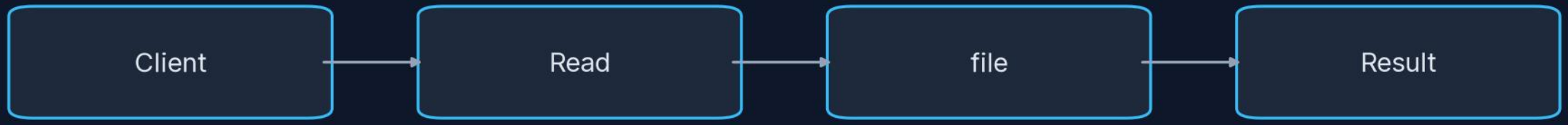
Today's Hook

- Problem: Read file → call service → write result; prod vs test interpreters
- Pattern: Effect Interpreters — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Read file → call service → write result; prod vs test interpreters
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Effect Interpreters: Read file → call service → write result; prod vs t

Real-World Scenario

- Running example: Read file → call service → write result; prod vs test interpreters
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.io.problem;

import java.util.Map;

/** Naive: effects inlined — cannot swap FS/HTTP for tests. */
public final class NaivePipeline {
    public static String run(Map<String, String> fs, String in, String out) {
        String content = fs.get(in);
        if (content == null) throw new IllegalStateException("missing");
        String resp = "OK:" + content; // hard-wired HTTP
        fs.put(out, resp);
        return resp;
    }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.io.problem;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.function.Function;

/**
 * Effect program: read a file → call a service → write a result.
 * Same AST runs under ProdInterpreter (in-memory FS+HTTP) and TestInterpreter (scripted).
 */
public final class EffectProgram {
    private EffectProgram() {}

    public sealed interface Eff permits ReadFile, HttpGet, WriteFile, Pure {}
    public record ReadFile(String path) implements Eff {}
    public record HttpGet(String url, String body) implements Eff {} // body = request payload from file
    public record WriteFile(String path, String content) implements Eff {}
    public record Pure(String value) implements Eff {}

    public abstract static class Prog<A> {
        private Prog() {}
        public abstract <B> Prog<B> flatMap(Function<? super A, Prog<B>> f);
        public final <B> Prog<B> map(Function<? super A, ? extends B> f) {
            return flatMap(a -> pure(f.apply(a)));
        }
```

Solution (2)

```
    public static <A> Prog<A> pure(A a) { return new Return<>(a); }
    public static Prog<String> readFile(String path) { return new Suspend<>(new ReadFile(path)); }
    public static Prog<String> httpGet(String url, String body) { return new Suspend<>(new HttpGet(url, body)); }
    public static Prog<Void> writeFile(String path, String content) { return new Suspend<>(new WriteFile(path, content)); }
}

private static final class Return<A> extends Prog<A> {
    final A value; Return(A value) { this.value = value; }
    public <B> Prog<B> flatMap(Function<? super A, Prog<B>> f) { return f.apply(value); }
}
private static final class Suspend<A> extends Prog<A> {
    final Eff eff; Suspend(Eff eff) { this.eff = eff; }
    public <B> Prog<B> flatMap(Function<? super A, Prog<B>> f) { return new Bind<>(this, f); }
}
private static final class Bind<A, B> extends Prog<B> {
    final Prog<A> prev; final Function<? super A, Prog<B>> f;
    Bind(Prog<A> prev, Function<? super A, Prog<B>> f) { this.prev = prev; this.f = f; }
    public <C> Prog<C> flatMap(Function<? super B, Prog<C>> g) {
        return new Bind<>(prev, a -> f.apply(a).flatMap(g));
    }
}
```

Solution (3)

```
    public static Prog<String> reportPipeline(String inPath, String url, String outPath) {
        return Prog.readFile(inPath).flatMap(content ->
                Prog
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.io.problem;

import java.util.HashMap;
import java.util.Map;

public final class ProblemDemo {
    public static void main(String[] args) {
        System.out.println("=== Effect Interpreters: read → HTTP → write ===");
        System.out.println("--- naive ---");
        var fs = new HashMap<String, String>();
        fs.put("/in.txt", "order-42");
        System.out.println(NaivePipeline.run(fs, "/in.txt", "/out.txt"));

        System.out.println("--- prod interpreter ---");
        var pfs = new HashMap<String, String>();
        pfs.put("/in.txt", "order-42");
        var prod = new EffectProgram.ProdInterpreter(pfs, body -> "SHIPPED:" + body);
        System.out.println(prod.run(EffectProgram.reportPipeline("/in.txt", "https://ship", "/out.txt")));
        System.out.println("log=" + prod.log());
        System.out.println("out=" + prod.fs().get("/out.txt"));

        System.out.println("--- test interpreter ---");
        var test = new EffectProgram.TestInterpreter(Map.of("/in.txt", "order-7"), "OK", false);
        System.out.println(test.run(EffectProgram.reportPipeline("/in.txt", "https://ship", "/out.txt")));
        System.out.println("writes=" + test.writes());
```

Runnable main (2)

```
        System.out.println("--- test failure: http down ---");
        try {
            new EffectProgram.TestInterpreter(Map.of("/in.txt", "x"), "OK", true)
                    .run(EffectProgram.reportPipeline("/in.txt", "https://ship", "/out.txt"));
        } catch (IllegalStateException e) {
            System.out.println("caught=" + e.getMessage());
        }
    }
}
```

Walkthrough Output

```
=== Effect Interpreters: read → HTTP → write ===
--- naive ---
OK:order-42
--- prod interpreter ---
SHIPPED:order-42
log=[read:/in.txt, http:https://ship, write:/out.txt]
out=SHIPPED:order-42
--- test interpreter ---
OK
writes={/out.txt=OK}
--- test failure: http down ---
caught=http-down
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Effect Interpreters.

Related Patterns

- Contrast Effect Interpreters with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Effect Interpreters → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.